Recovery Contracts for Post-Quantum Storage Migration

Supplementary Material and Reproduction Guide

Ishan Teredesai Tushar Rahangdale

Author-review draft, 7 November 2026

1 Scope of the evidence

This supplement accompanies the four-page workshop draft. The evidence consists of an abstract key-reference model and a synthetic SQLite/AES-GCM crash harness. Neither implements ML-KEM, a hybrid KEM, a hardware QRNG, or a production KMS. The observations establish selected dependency failures and the behavior of a guarded retirement rule under the declared assumptions. They are not a cryptographic security proof, a performance benchmark, or a statistical estimate of incident frequency.

2 Reproduction

The recorded environment was Python 3.12.14, SQLite 3.53.1, cryptography 50.0.2, and Linux x86-64. Use Python 3.10 or newer on Linux; Windows users can use WSL. The model itself uses only the Python standard library. The crash harness needs the pinned dependency in `requirements.txt`.

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
bash run_experiments.sh
```

The script runs validation tests, explores all eighteen model configurations, and executes all 126 harness cases. Outputs replace files under `results/`; save earlier copies to compare environments. Temporary ciphertexts and local synthetic keys are deleted after each case. All data are synthetic; no user records or external cloud services are accessed.

File	Contents
`experiment/model.py`	State definition, transitions, retirement guards, breadth-first search.
`experiment/crash_harness.py`	Synthetic encryption, separate key inventory, worker termination, exact plaintext restoration.
`experiment/test_model.py`	Six validation tests, including invariant preservation over a small structurally valid universe.
`results/model_summary.csv`	All eighteen model configurations.
`results/model_traces.json`	Shortest witnesses with every intermediate state.
`results/crash_trials.csv`	Individual outputs for all 126 harness cases.
`results/*manifest.json`	Exact runtime versions, assumptions, and source hashes.
`SHA256SUMS`	Integrity hashes of code, results, sources, and PDFs.

3 How to interpret a successful run

All six tests should pass. The model's `recovery_set` policy should have zero states with missing live or backup keys. In the crash harness it should restore every live and backup record in all 42 assigned cases. Intentionally weak control policies should fail in the documented cases. A run that gives all policies zero failures has not reproduced the reported experiment. Environment manifests may differ between machines; the policy outcome counts should agree.

4 Transition system and complete model results

A state is (a, L, B, P, K): active generation, committed live references, optional snapshot references, volatile pending targets, and durable available keys. All live and snapshot references start at zero; the only initial key is zero. At most one snapshot is retained. Each live record has at most one pending target. Generations are provisioned in increasing order and retired identifiers are never reused.

Transition	Precondition	Effect
Activate	A next generation is within the bound.	Add its durable key; change active generation.
Prepare record i	No pending target; live generation differs from active; source key exists.	Set volatile pending target to the active generation.
Commit record i	Pending target exists and its key is available.	Atomically replace committed reference; clear its pending target.
Snapshot	No retained snapshot; live keys exist.	Copy live references to one durable snapshot.
Release snapshot	Snapshot exists.	Remove that retention obligation.
Crash	At least one pending target exists.	Clear all pending targets; retain durable state.
Retire key k	Key is available, not active, and the selected policy finds no dependency.	Remove key from the durable available set.

Breadth-first exploration continues to a fixed point without a depth/state cutoff. Each state is processed once per configuration. A transition count includes every generated outgoing edge, including edges to an already visited state. Backup release and recreation may create cycles. The last column counts states whose live references are target-labeled while a retained snapshot still has a legacy reference. None of these counts is a probability.

Records	Generations	Policy	States	Edges	Live loss	Backup loss	Profile gap
1	2	intent-only	16	31	4	3	2
1	2	live-only	12	23	0	1	2
1	2	recovery-set	11	21	0	0	1
1	3	intent-only	76	202	26	20	14
1	3	live-only	52	137	0	7	10
1	3	recovery-set	43	111	0	0	5
2	2	intent-only	48	115	16	13	6
2	2	live-only	32	79	0	3	6
2	2	recovery-set	29	73	0	0	3
2	3	intent-only	500	1,642	230	202	126
2	3	live-only	288	955	0	55	82
2	3	recovery-set	217	713	0	0	39
3	2	intent-only	154	448	52	45	14
3	2	live-only	102	319	0	7	14
3	2	recovery-set	95	305	0	0	7
3	3	intent-only	3,682	14,499	1,814	1,766	926
3	3	live-only	1,996	8,031	0	403	610
3	3	recovery-set	1,495	6,009	0	0	293

Loss means a missing key reference. Live-loss and backup-loss columns may overlap. The recovery-set totals, summed across its six configurations, are 1,890 states, 7,232 edges, and zero states with either missing-key condition. States counted in separate configurations are not a union of distinct deployment states.

5 Minimal witnesses and invariant interpretation

Write $[x]$ for a one-record reference vector and $-$ for no pending target. The initial state is $(0, [0], [0], [-], \{0\})$.

Action	Durable consequence and failure mechanism
Activate 1	$a = 1$, $K = \{0, 1\}$; both copies still reference 0.
Prepare record 0 to 1	$P = [1]$ but $L = [0]$ remains committed.
Intent-only retirement of 0	$K = \{1\}$; committed live and backup references still require 0. A crash clears P.
Alternative: commit first	$L = [1]$, $B = [0]$, $K = \{0, 1\}$.
Live-only retirement of 0	Live data still has its key; the snapshot no longer has its key.
Recovery-set decision	Reject retirement while $B = [0]$ remains a required restore point.

The reference invariant is $\{L_i\} \cup \{B_i\} \subseteq K$. Pending work is not a durable substitute for the old committed reference. The guarded policy also protects pending target keys to avoid invalidating prepared work. Its preservation argument relies on serialized transitions, complete dependency information, and durable provisioning before activation. It is not a proof of an implementation that scans a stale inventory while other writers run.

Profile coverage remains separate. Immediately after a successful live commit, $L = [1]$ and $B = [0]$ can both be recoverable. The live profile has migrated while the snapshot profile has not. Releasing the snapshot makes that particular obligation disappear; the experiment does not establish that this is a legally or operationally acceptable retention decision.

6 Crash harness and cut points

Each record has an independently generated 32-byte DEK. Payload encryption and DEK wrapping use AES-256-GCM with separate random 12-byte nonces. Data AAD binds the record identifier and content version; wrapping AAD binds identifier and key epoch. The local key inventory is separate from the SQLite transaction. The harness takes a snapshot before migrating the live envelopes from epoch zero to one. Six cut points terminate the worker with exit code 93; normal completion exits with zero.

For every cell below, L/B is the number of cases that fail live/backup recovery, out of six synthetic datasets. Dataset sizes are one or eight records at 512 B, 16 KiB, or 256 KiB each. Each policy/cut/dataset is executed once.

Cut point / completion	Intent-only L/B	Live-only L/B	Recovery-set L/B
After durable key provisioning	0/0	0/0	0/0
After replacement envelopes are prepared	0/0	0/0	0/0
After first retirement attempt	6/6	0/0	0/0
After SQL updates, before commit	6/6	0/0	0/0
After database commit	0/6	0/0	0/0
After final retirement attempt	0/6	0/6	0/0
Normal completion	0/6	0/6	0/0

There are 126 cases, including 108 worker terminations. All observed failure cases are missing-key dependencies. Authentication failures, unexpected record counts, and incorrect recovered bytes cause the runner to abort instead of silently counting them as the expected dependency failure.

7 Validation, provenance, and remaining work

The six validation tests check: a lost snapshot under live-only retirement; lost live data after premature retirement and a crash; guarded retirement after an explicit snapshot release; the distinction between profile coverage and recoverability; invariant preservation over a small structurally valid state universe; and detection of the intentionally weak controls by the explorer. The preservation test checks all generated guarded transitions from valid states with two records and at most three generations (including unreachable states). The supplied test run passes. This does not certify the implementation of a real key-management service.

7.1 Threats to validity

- The two weak policies are purpose-built negative controls. The experiment is not evidence of a vulnerability in AWS KMS, MongoDB, or another deployed product.
- The model assumes a complete inventory, serialized steps, and no ordinary record content updates/deletes. A distributed system needs a concurrency and reconciliation protocol for the reference check and key retirement.
- One retained snapshot omits logs, replicas, disconnected copies, and multi-level recovery chains. The guard must be extended to cover their actual dependencies before it supports their retention obligations.
- Process termination is not power loss. No kernel crash, torn sector, faulty drive, filesystem reordering campaign, or remote KMS outage was tested.
- Fixed tiny datasets and a single environment establish neither throughput nor scalability. Outcome counts are deterministic case coverage, not incident frequencies or a reliability confidence interval.
- Labels for legacy and target profiles are abstract. Actual KEM integration, authenticated format negotiation, authorization, parser compatibility, compromise recovery, and post-quantum security require separate validation.

7.2 Suggested follow-up on the authors' machine

First reproduce the supplied counts unchanged. Next, integrate the reference check with the actual middleware and key store, while preserving a synthetic backup that still depends on the previous epoch. Inject interruption before and after durable envelope commit and retirement, then restore both copies and compare bytes. Record the exact application revision, KMS configuration, database version, snapshot policy, and failure injection mechanism. These integration results must be reported as a new experiment rather than attributed to the present reference harness.

7.3 Provenance and publication status

OpenAI ChatGPT assisted with model design, code generation, validation code, and drafting. The experiment programs were executed to produce the included outputs. The listed authors must review and adopt the work before submission. A separate confidential author note identifies a related four-author manuscript, explains the contribution boundary, and records that its withdrawal/publication status has not been verified. This artifact does not resolve that status or provide assurance against duplicate-submission or overlapping-publication issues. The main paper is self-contained; this supplement adds reproduction detail.

7.4 Integrity

The JSON manifests record SHA-256 hashes of the exact model and harness source used for the reported results. The archive's `SHA256SUMS` covers the final source files and outputs. To check it from the artifact root, run:

```
sha256sum --check SHA256SUMS
```

The comparison may fail after deliberately rerunning on a different environment because runtime manifests change. Keep the delivered archive as the original record and rerun in a separate copy when comparing environments.